Instituto Tecnológico de Costa Rica

Escuela de Administración de Tecnologías de Información



Entregable Tarea #3

Bases de Datos Avanzadas

Profesor:

Manuel Zumbado Corrales

Por:

Emmanuel Esquivel Chavarría – 2022312336

Yerik Chaves Serrano – 2022437493

Jose Pablo Guerrero Duarte – 2022099311

Octubre, 2026

1. Parte A: consultas sobre `boleteria.eventos`

Las consultas se ejecutaron sobre **400 eventos** en MongoDB 8.0 (un solo nodo). El script completo y sus salidas se encuentran en `labs/lab3-mongo-replicas/tarea/consultas.js` y `evidence/tarea3/consultas.txt`.

1.1 A1. Conciertos de Heredia por fecha

Se filtraron conciertos de Heredia del 01/12/2026 al 28/02/2027, proyectando únicamente nombre, recinto y fecha en orden ascendente.

```
db.eventos.find({categoria:"concierto", ciudad:"Heredia",
  fecha:{$gte:"2026-12-01",$lte:"2027-02-28"}},
  {_id:0,nombre:1,recinto:1,fecha:1}).sort({fecha:1})
```

Resultado: 7 eventos. Se comparó `explain(.executionStats")` antes y después de crear el índice `{categoria:1, ciudad:1, fecha:1}`.

Métrica	Sin índice	Con índice
`totalDocsExamined`	400	7
`totalKeysExamined`	0	7
`nReturned`	7	7
Etapa de acceso	`COLLSCAN`	`IXSCAN`
Ordenamiento explícito	`SORT`	No

El índice ubica primero los campos con **igualdad** (`categoria`, `ciudad`) y después el campo de **rango y ordenamiento** (`fecha`). La cantidad de documentos examinados se redujo un **98,25 %**; el índice también suministró el orden solicitado.

1.2 A2. Zona VIP con precio menor a ₡48 000

Se usó `$elemMatch` tanto en el filtro como en la proyección para garantizar que el nombre VIP y el precio correspondieran a la *misma* zona y devolver únicamente esa zona.

```
db.eventos.find({zonas:{$elemMatch:{nombre:"VIP",
  precio:{$lt:48000}}}}, {_id:0,nombre:1,
  zonas:{$elemMatch:{nombre:"VIP",precio:{$lt:48000}}}})
```

Comparación: condiciones independientes sobre `zonas.nombre` y `zonas.precio`: **265** eventos; usando `$elemMatch`: **195**. Los **70 falsos positivos** se deben a que, sin `$elemMatch`, ambas condiciones pueden satisfacerse en elementos distintos del arreglo.

1.3 A3. Eventos accesibles, excluyendo categorías

```
db.eventos.find({etiquetas:"accesible",
 categoria:{$nin:["teatro","conferencia"]}},
 {_id:0,nombre:1,categoria:1,etiquetas:1})
```

Resultado: 91 eventos. MongoDB busca la etiqueta dentro del arreglo y `$nin` excluye ambas categorías.

1.4 A4. Eventos sin patrocinador por categoría

```
db.eventos.aggregate([
 {$match:{patrocinador:{$exists:false}}},
 {$group:{_id:"$categoria",cantidad:{$sum:1}}},
 {$sort:{cantidad:-1}}
])
```

Conferencia	61	Deporte	58	Concierto	57
Teatro	50	Festival	49	**Total**	**275**

`$exists:false` identifica documentos donde el campo `patrocinador` no existe; `$group` cuenta por categoría.

2. Parte A: consultas (continuación)

2.1 A5. Capacidad total de zonas por ciudad

```
db.eventos.aggregate([
 {$unwind:"$zonas"},
 {$group:{_id:"$ciudad",capacidadTotal:{$sum:"$zonas.capacidad"}}},
 {$sort:{capacidadTotal:-1}}
])
```

Ciudad	Capacidad	Ciudad	Capacidad
Heredia	30 380	Puntarenas	17 180
Liberia	24 160	San José	17 150
San Carlos	21 810	Limón	15 560
Alajuela	21 020	Cartago	15 550

`$unwind` separa las zonas de cada evento; `$group` suma las capacidades por ciudad. **Heredia** registra la mayor capacidad acumulada (30 380).

3. Parte B: ¿anidar o referenciar órdenes?

1. Tamaño BSON y límite. Se midió una orden con `_id`, `evento_id`, `zona`, `cantidad`, `estado` y `creada_en` mediante `bsonsize()`, con un resultado de **118 bytes**. Para reproducir la medición se utilizó el siguiente objeto BSON:

```
bsonsize({_id:"ORD-000001",evento_id:"EV-0001",
 zona:"VIP",cantidad:2,estado:"confirmada",
 creada_en:ISODate("2026-10-28T12:00:00Z")})
```

Considerando el límite de 16 MiB por documento:

$$N_{\text{ideal}} = \left\lfloor \frac{16 \times 1024 \times 1024}{118} \right\rfloor = \boxed{142\,179 \text{ órdenes}}$$

Este cálculo es una **cota idealizada**, no una capacidad garantizada: el evento ocupa espacio y el arreglo añade sobrecarga por las claves de posición; además, el tamaño real de las órdenes puede variar.

2. Escrituras concurrentes. Anidar zonas y órdenes permite actualizar información relacionada de forma atómica dentro de un documento, simplificando la consistencia. Como desventaja, cientos de compradores modificando el mismo evento generan contención sobre ese documento, además del crecimiento continuo hacia el límite BSON. Por ello, resulta más escalable guardar las órdenes en una **colección independiente** con `evento_id` como referencia.

3. Regla de modelado. Conviene *anidar* datos acotados, estrechamente relacionados y consultados juntos; conviene *referenciar* datos de crecimiento elevado o con ciclo de vida y consultas independientes. En un sistema universitario, los contactos del estudiante pueden anidarse, mientras que sus matrículas se almacenan aparte y referencian al estudiante.

Trazabilidad: todas las cifras provienen de `evidence/tarea3/consultas.txt`. **Repositorio:** github.com/Yerik1/Laboratorio-3-BDA.